OPERATING SYSTEMS

Memory Management

Pavan A C
Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Virtual Memory

Pavan A C
Department of Computer Science & Engineering

Background (Cont.)

- **Virtual address space** – logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

PES UNIVERSITY
CELEBRATING 50 YEARS

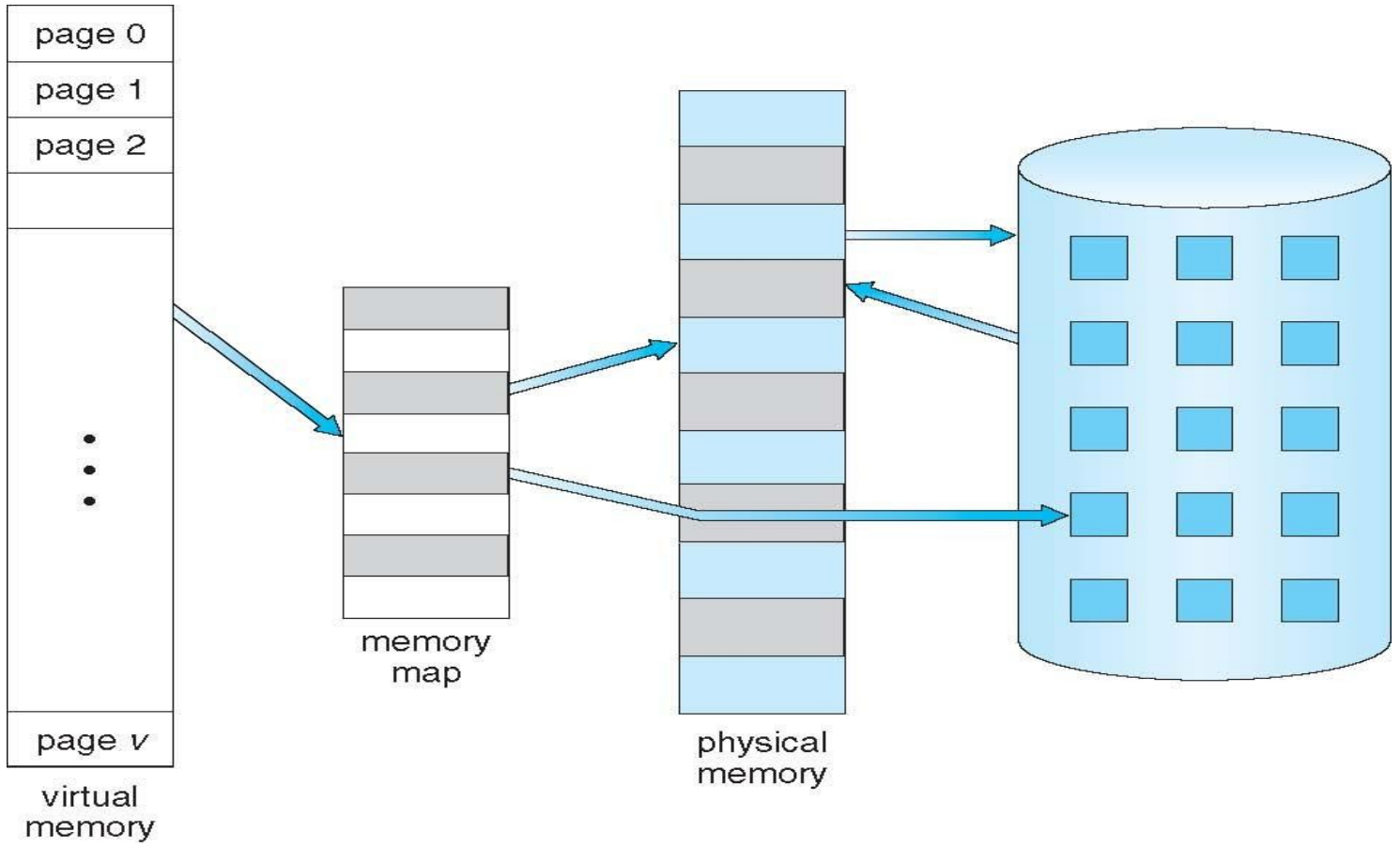
Background



- Virtual memory allows programs larger than physical memory
- Only part of program needs to be in memory
- Logical memory separated from physical memory
- More programs can run concurrently

OPERATING SYSTEMS

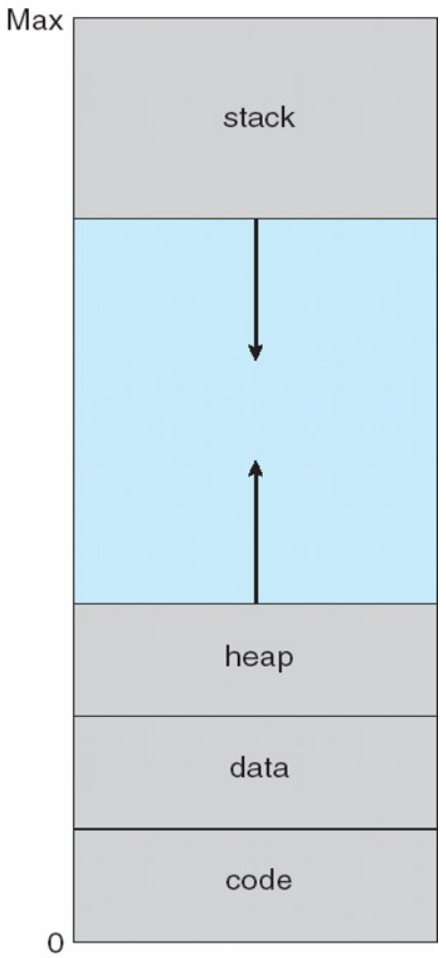
Virtual Memory That is Larger Than Physical Memory



OPERATING SYSTEMS

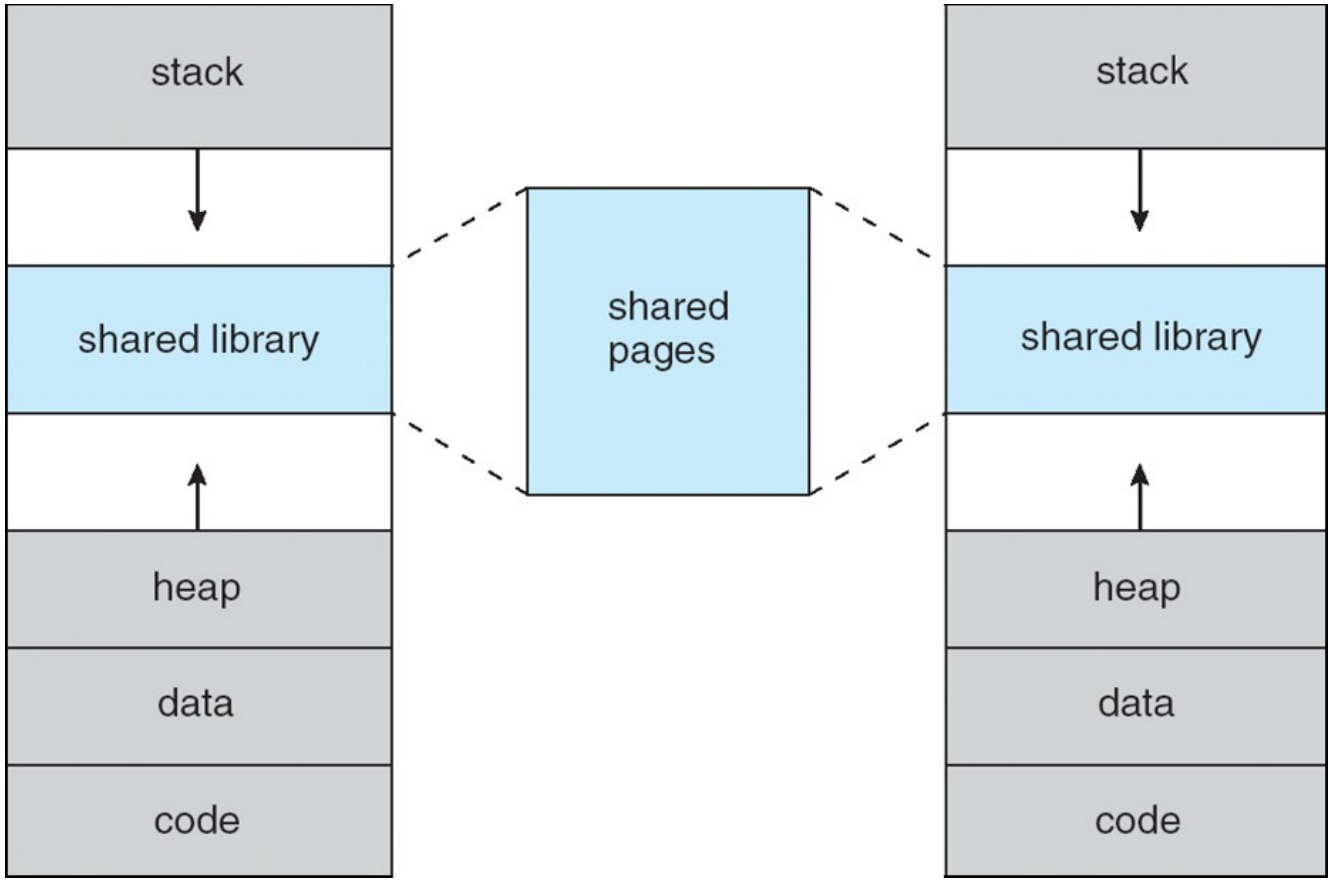
Virtual-address Space

- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
  - Maximizes address space use
  - Unused address space between the two is hole
    - 4 No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during fork(), speeding process creation



PES UNIVERSITY
CELEBRATING 50 YEARS

Shared Library Using Virtual Memory

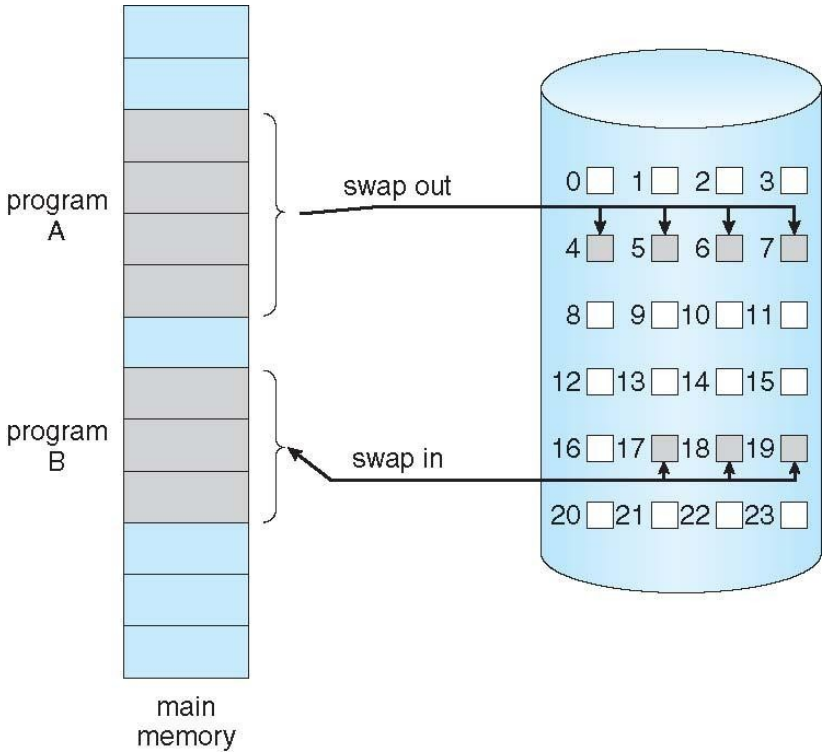


PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Demand Paging

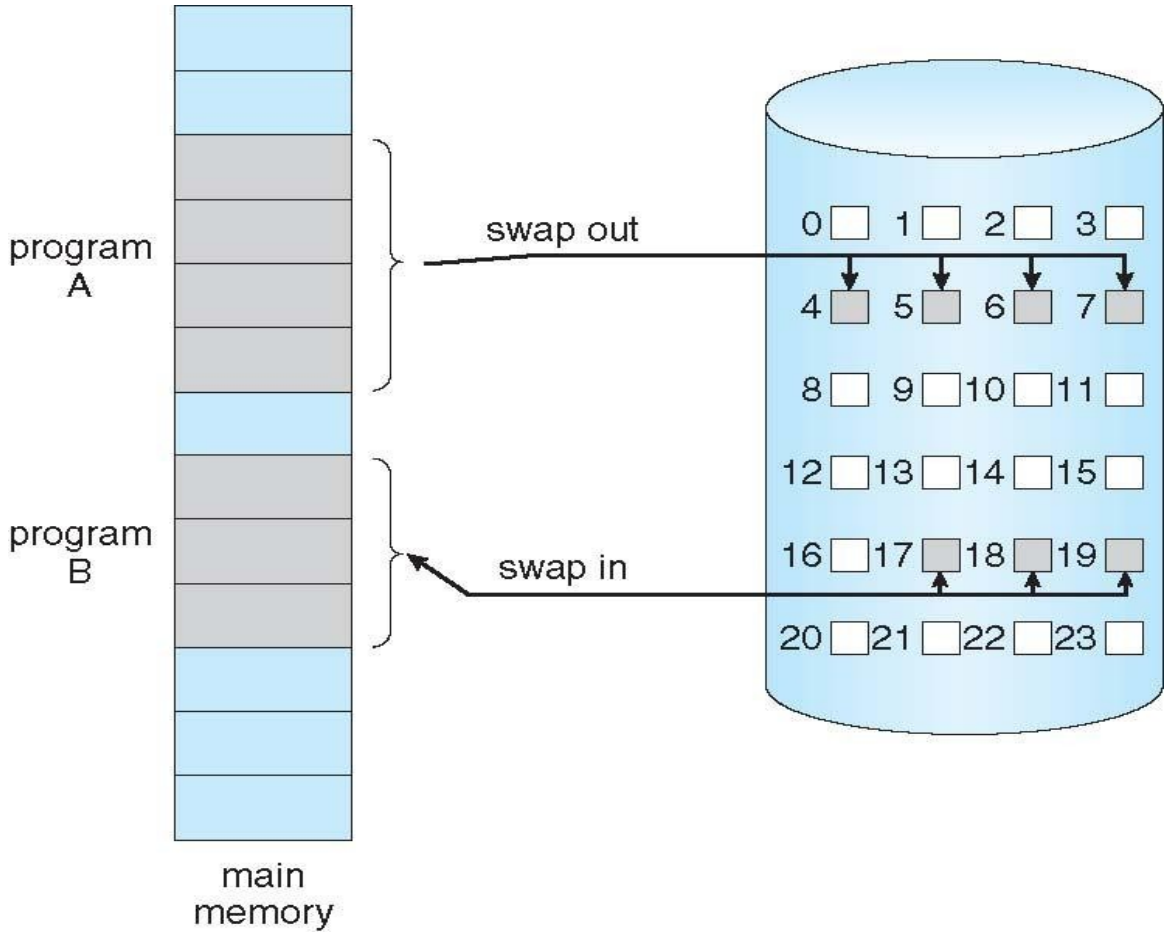
- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping (diagram on right)
- Page is needed ⇒ reference to it
  - invalid reference ⇒ abort
  - not-in-memory ⇒ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is called a **pager**



PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Demand Paging



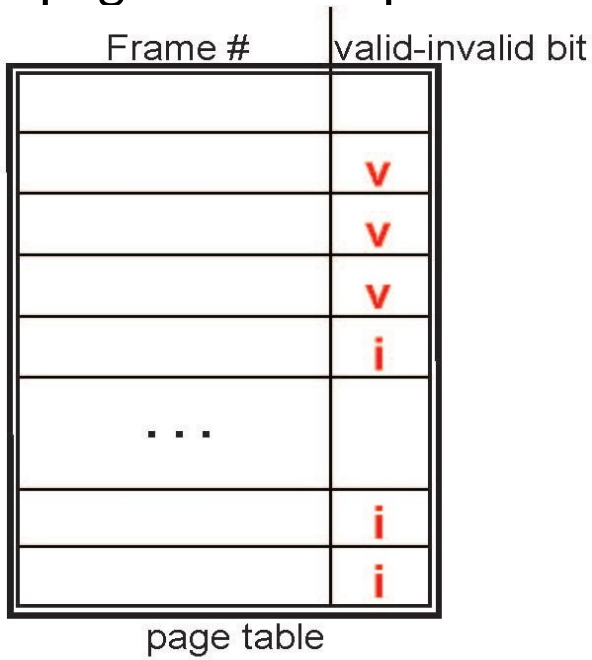
Demand Paging – Basic Concepts



- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- How to determine that set of pages?
  - Need new MMU functionality to implement demand paging
- If pages needed are already **memory resident**
  - No difference from non demand-paging
- If page needed and not memory resident
  - Need to detect and load the page into memory from storage
    - Without changing program behavior
    - Without programmer needing to change code

Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:

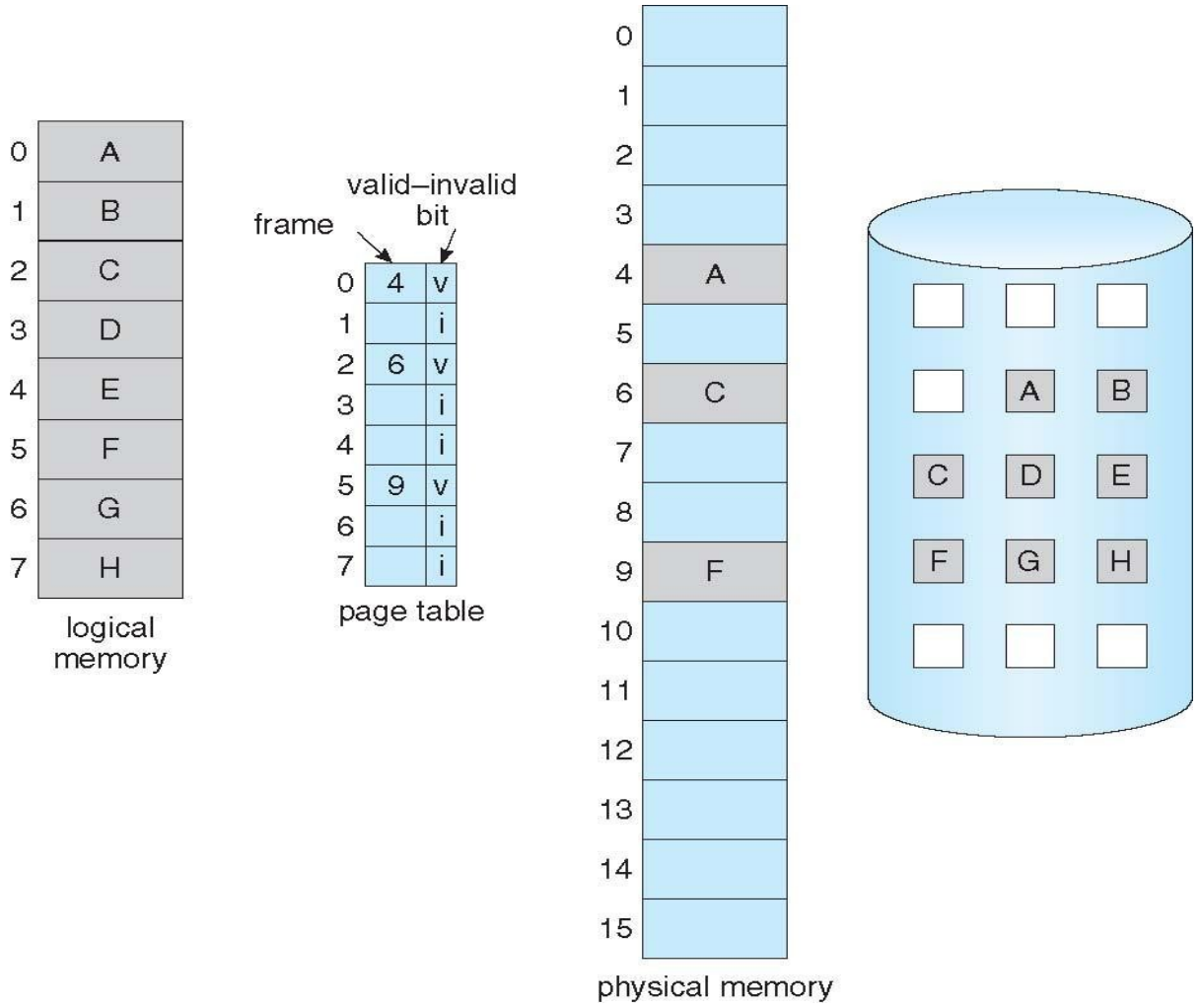


- During MMU address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Page Table When Some Pages Are Not in Main Memory



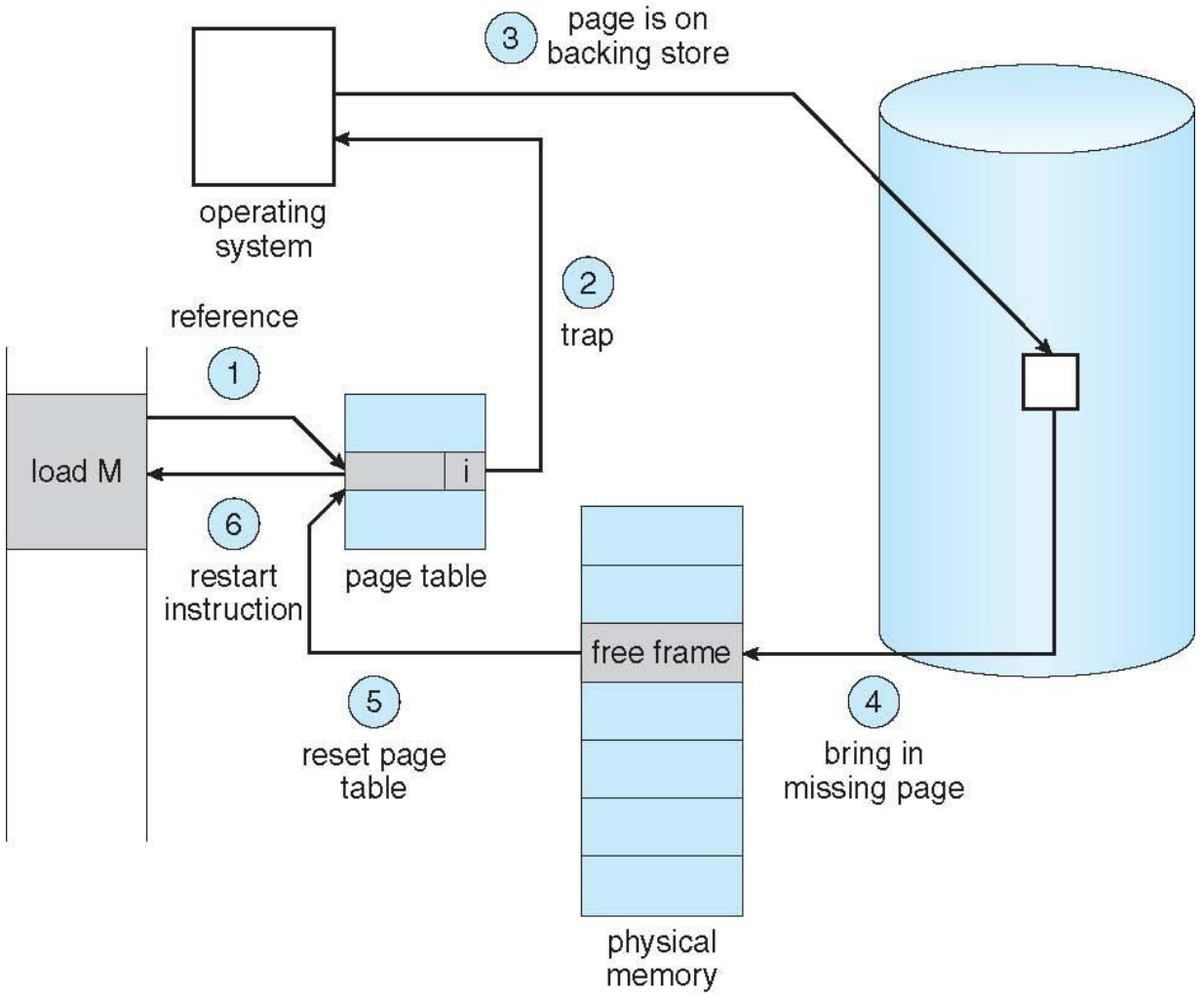
Steps in Handling Page Fault

1. If there is a reference to a page, first reference to that page will trap to operating system
   - **Page fault**
2. Operating system looks at another table to decide:
   - Invalid reference ⇒ abort
   - Just not in memory
3. Find free frame
4. Swap page into frame via scheduled disk operation
5. Reset tables to indicate page now in memory
   Set validation bit = **v**
6. Restart the instruction that caused the page fault

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Steps in Handling Page Fault (Cont.)



Major vs Minor page faults

PES UNIVERSITY
CELEBRATING 50 YEARS

- Major page fault: the page we want to reference exists only on the system disk, thus reading it will be an expensive operation. This type is also called “Hard Page Fault”.
- Minor Page Fault: the required page is not stored on disk but in some other place in memory. An example for that is in the working set of another process. It is also sometimes called “Soft Page Fault”.

OPERATING SYSTEMS

Aspects of Demand Paging



- Extreme case – start process with *no* pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
  - And for every other process pages on first access
  - **Pure demand paging**
- Actually, a given instruction could access multiple pages -> multiple page faults
  - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
  - Pain decreased because of **locality of reference** - Process migrates from one locality (i.e., a set of pages that are actively used together) to another
- Hardware support needed for demand paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with **swap space**)
  - Instruction restart

OPERATING SYSTEMS

Instruction Restart

- Consider an instruction that could access several different locations (Ex: move some bytes from one location to another possibly overlapping location)
  - Source and destination blocks overlap i.e straddle a page boundary
    - Page fault might occur after the move is partially done
    - Source block may have been modified so we cannot simply restart the instruction
    - In one solution, the microcode computes and attempts to access both ends of both blocks.
      - Page fault can occur before anything is modified
    - The other solution uses temporary registers to hold the values of overwritten locations.
      - If Page fault occurs, all the old values are written back into memory before the trap occurs

OPERATING SYSTEMS

Performance of Demand Paging

- Stages in Demand Paging (worse case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
    a) Wait in a queue for this device until the read request is serviced
    b) Wait for the device seek and/or latency time
    c) Begin the transfer of the page to a free frame

PES UNIVERSITY
CELEBRATING 50 YEARS

Performance of Demand Paging (Cont.)

6. While waiting, allocate the CPU to some other user
6. Receive an interrupt from the disk I/O subsystem (I/O completed)
6. Save the registers and process state for the other user
6. Determine that the interrupt was from the disk
6. Correct the page table and other tables to show page is now in memory
6. Wait for the CPU to be allocated to this process again
6. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Performance of Demand Paging (Cont.)

- Three major activities
  - Service the interrupt – careful coding means just several hundred instructions needed
  - Read the page – lots of time
  - Restart the process – again just a small amount of time
- Page Fault Rate $0 \le p \le 1$ (p is the probability of a page fault)
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

EAT = (1 – *p*) x memory access

+ *p* x page-fault service time

OPERATING SYSTEMS

Demand Paging Example



- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- EAT = (1 – p) x 200 + p (8 milliseconds)

  = (1 – p) x 200 + p x 8,000,000

  = 200 + p x 7,999,800

- EAT is directly proportional to page-fault rate
- If one access out of 1,000 causes a page fault i.e p = 0.001, then

  EAT = 8200 ns or 8.2 microseconds.

  This is a slowdown by a factor of 40!! (i.e. 8200/200)
- If we want performance degradation < 10 percent
  - 220 > 200 + 7,999,800 x p

    20 > 7,999,800 x p
  - p < .0000025
  - < one page fault in every 400,000 memory accesses

Demand Paging Optimizations (Cont.)



- Still need to write to swap space
  - Pages not associated with a file (like stack and heap) – **anonymous memory**
  - Pages modified in memory but not yet written back to the file system
- Mobile systems:
- Modern Android and macOS/iOS heavily use compressed memory (e.g., ZRAM).
- Instead of swapping to disk, the OS compresses the inactive page and leaves it in RAM, treating CPU cycles as cheaper than disk I/O.

OPERATING SYSTEMS

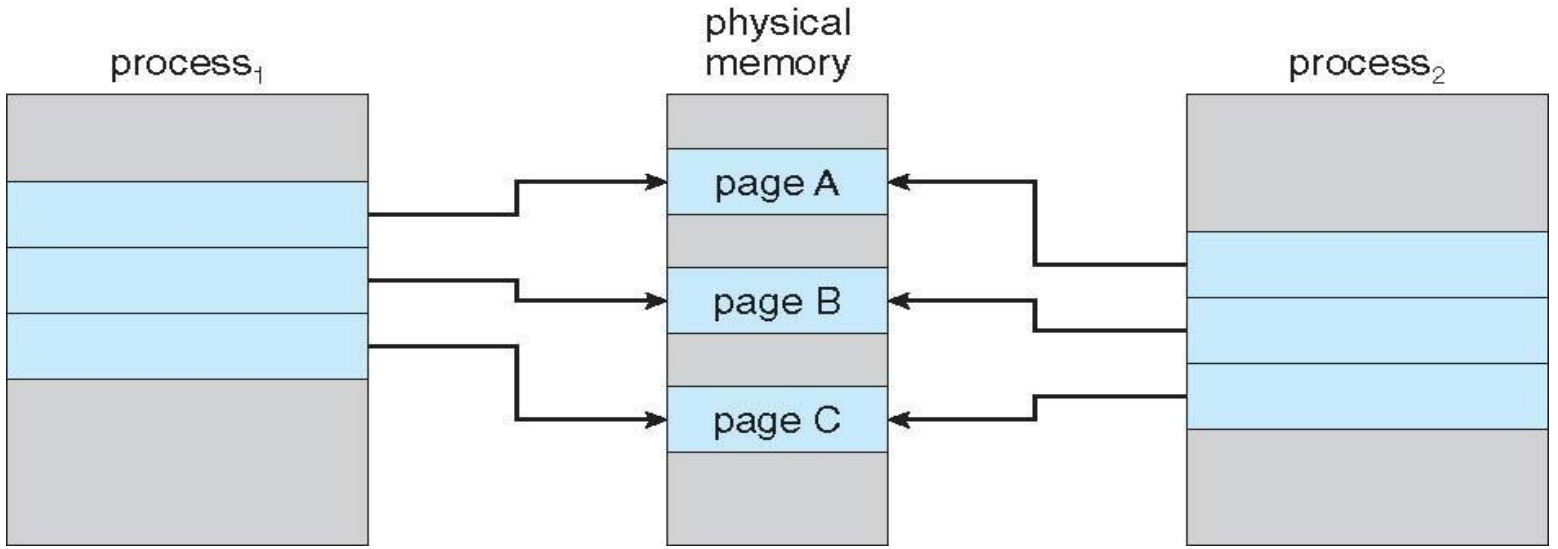
Copy-on-Write

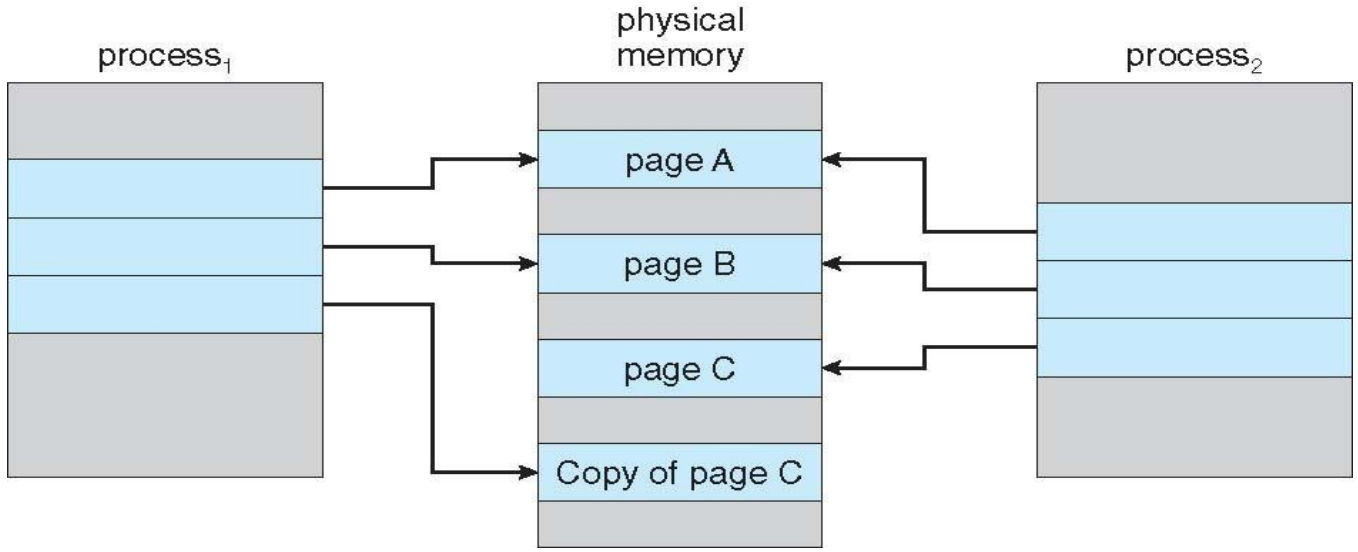
- Copy-on-Write (COW) allows both parent and child processes to initially ***share*** the same pages in memory
  - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- In general, free pages are allocated from a pool of zero-fill-on-demand pages
  - Pool should always have free frames for fast demand page execution
    - Don't want to have to free a frame as well as other processing on page fault
- **Hardware Enforcement:** The OS marks the page table entries for *both* processes as **Read-Only**.
- **The Trap:** If either process attempts to modify the data, the MMU triggers a hardware protection fault. The OS intercepts this trap, realizes it's a COW page, allocates a new physical frame, copies the data, updates the page table to Read/Write, and restarts the instruction.

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Before and After Process 1 Modifies Page C





What Happens if There is no Free Frame?



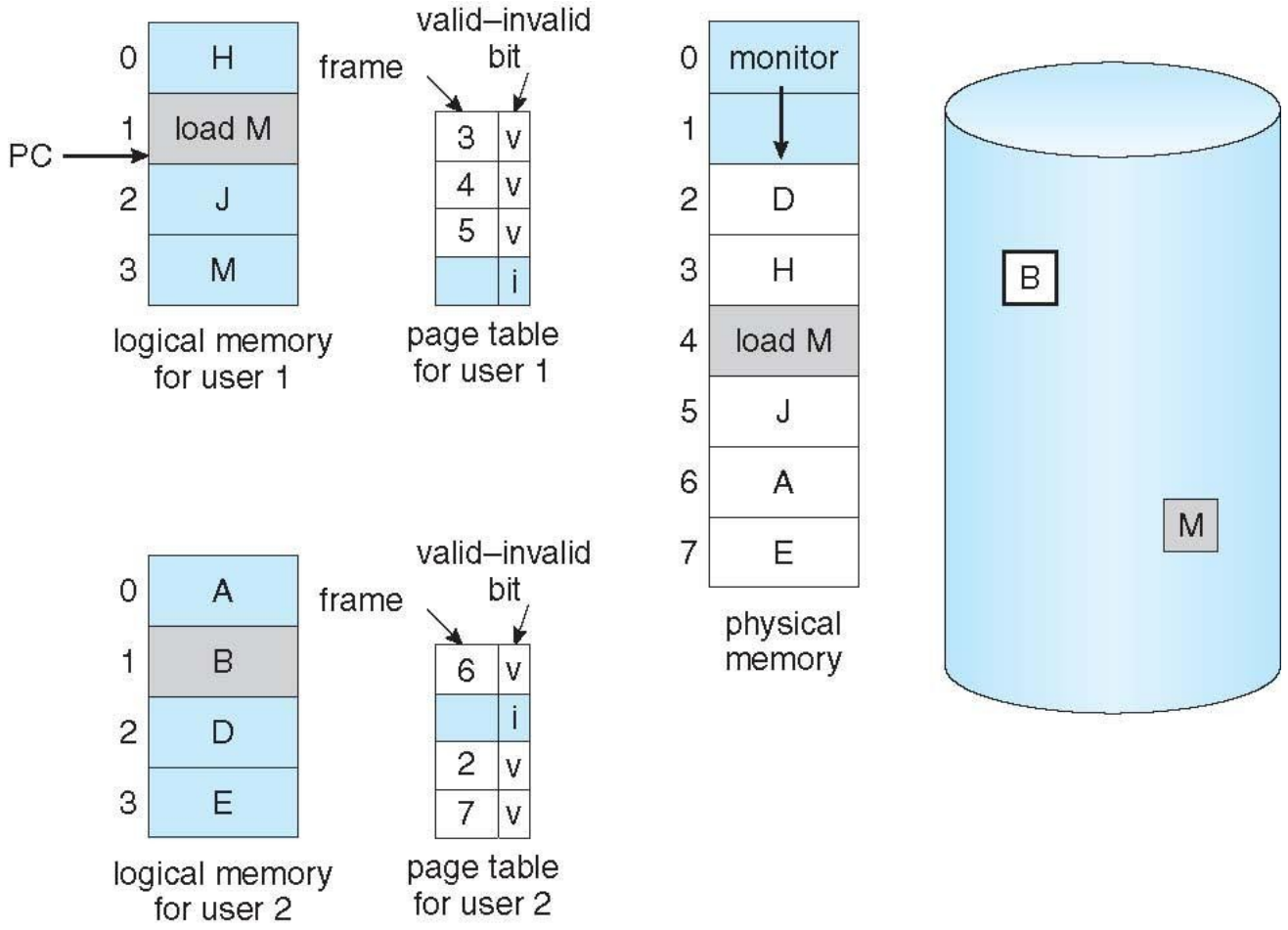
- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
  - Algorithm – terminate? swap out? replace the page?
  - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement



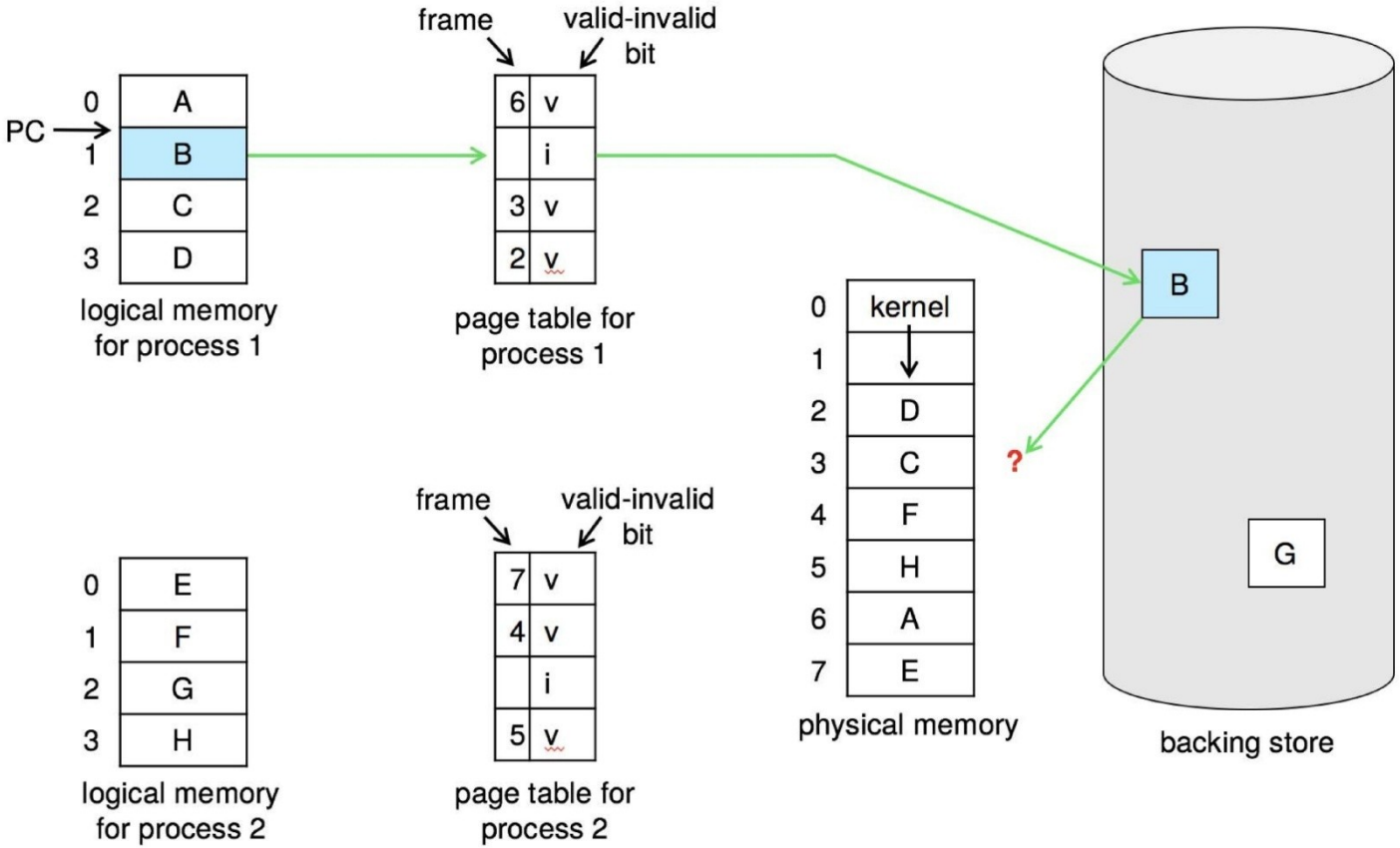
- Prevent **over-allocation** of memory by modifying page-fault service routine to include page replacement (vs simply increasing degree of multiprogramming)
- Use **modify** (**dirty**) **bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory

Need For Page Replacement – Example 1



OPERATING SYSTEMS

Need For Page Replacement –Example 2



Basic Page Replacement

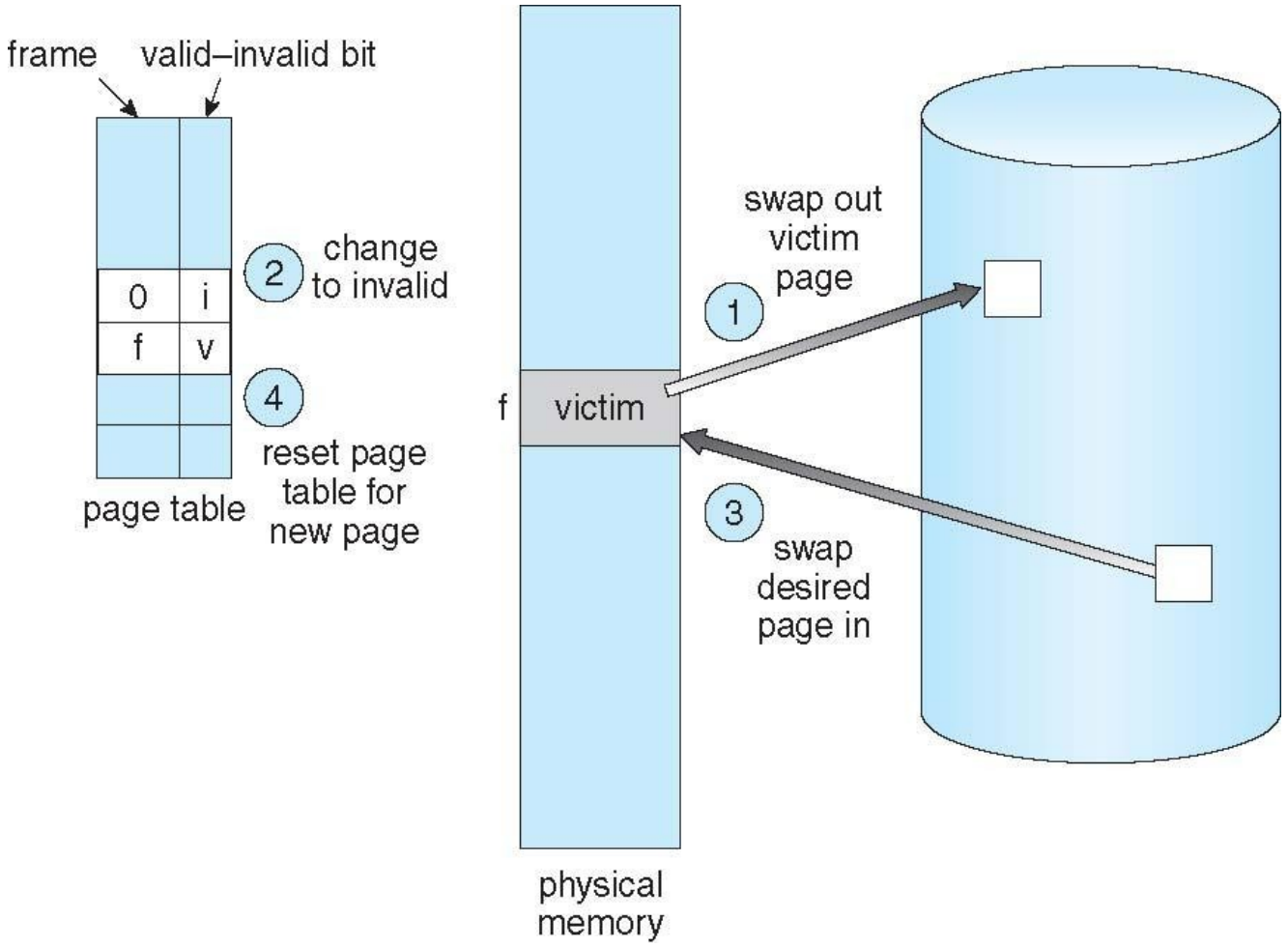


1. Find the location of the desired page on disk
2. Find a free frame:
   - If there is a free frame, use it
   - If there is no free frame, use a page replacement algorithm to select a **victim frame**
     - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Note now potentially 2 page transfers for page fault – increasing EAT

OPERATING SYSTEMS

Basic Page Replacement (Cont.)



Linux Angle



- **The Synchronous Problem:** Should OS wait till memory is 100% before invoking page replacement algorithms?.
- **The Linux Solution (kswapd):** Linux does not wait for memory to hit 0 free frames.
- **Watermarks:** OS maintains "high" and "low" watermarks. When free memory drops below the low watermark, kswapd wakes up in the background and proactively searches for inactive pages to evict, building up a pool of free frames so that programs rarely have to wait synchronously for a victim frame to be written to disk.

Linux Angle



- **The Synchronous Problem:** Should OS wait till memory is 100% before invoking page replacement algorithms?.
- **The Linux Solution (kswapd):** Linux does not wait for memory to hit 0 free frames.
- **Watermarks:** OS maintains "high" and "low" watermarks. When free memory drops below the low watermark, kswapd wakes up in the background and proactively searches for inactive pages to evict, building up a pool of free frames so that programs rarely have to wait synchronously for a victim frame to be written to disk.

OPERATING SYSTEMS

Problems

Q1: Given: Memory access time = 200 ns, Page fault rate = 0.001, Page fault service time = 8 ms Find the Effective Access time.

EAT=(1−0.001)×200+0.001×8,000,000

=199.8+8000 = EAT=8199.8 ns

Q2: Find Maximum Page Fault Rate if Memory access time is 100 ns, Page fault service time is 20 ms and Required EAT ≤ 120 ns

$$120 = 100 - 100p + 20{,}000{,}000p$$

$$20 = 19{,}999{,}900p$$

$$p = \frac{20}{19{,}999{,}900}$$

$$\boxed{p \approx 1 \times 10^{-6}}$$

OPERATING SYSTEMS

Questions

- Q1: What is the fundamental definition of Virtual Memory?
- Q2: In a virtual address space, why is it beneficial that the Stack and Heap grow towards each other from opposite ends?
- Q3: What does the term "Pure Demand Paging" mean?
- Q4: How does the Memory Management Unit (MMU) differentiate between a page that is in RAM and a page that is swapped to disk?
- Q5: What is the immediate hardware response when a CPU attempts to access a page marked with an 'i' (invalid) bit?
- Q6: Why is a "Minor Page Fault" much faster to resolve than a "Major Page Fault"?
- Q7: Describe the architectural difficulty of "Instruction Restart" during a page fault.
- Q8: What is the difference between File-backed memory and Anonymous memory?
- Q9: Why do mobile operating systems (like iOS/Android) typically avoid traditional swapping to a disk partition?

OPERATING SYSTEMS

Questions

- **Q1: In a Copy-on-Write (COW) system, what hardware permission does the OS apply to the shared pages immediately after a `fork()`?**
- **Q2: What specific problem necessitates the use of a Page Replacement algorithm?**
- **Q3: What is a "Victim Frame"?**
- **Q4: What is the purpose of the "Dirty Bit" (Modify bit) in a page table entry?**
- **Q5: Why does replacing a dirty victim page effectively double the page-fault service time?**
- **Q6: What must the OS do to the page table of the process whose page was just evicted as a victim?**
- **Q7: In modern systems like Linux, does the OS wait until there are exactly 0 free frames before executing page replacement?**



THANK YOU

Pavan A C
Department of Computer Science and Engineering

pavanac@pes.edu